

NiceGUI app.storage.tab 全面详细解析

`app.storage.tab` 是 NiceGUI 内置的五种存储类型之一，专为浏览器标签页级别的专属数据存储设计，核心特点是“单标签独立存储、跨页面重载 / 浏览器重启保留、服务器端内存存储”，适用于需要隔离不同标签页数据（如独立会话、专属状态）且无需长期持久化到磁盘的场景。

一、核心定位与设计目标

1. 核心价值

- 标签页隔离**：同一用户打开多个应用标签页时，每个标签页拥有独立的 `app.storage.tab` 存储，数据互不干扰（如标签页 1 的计数器、搜索条件不会影响标签页 2）；
- 跨重启保留**：只要标签页未关闭，即使应用服务器重启、浏览器重启（标签页恢复），存储的数据仍可保留（需等待官方 #2841 特性完全实现，当前已支持基础保留能力）；
- 服务器端存储**：数据存储在服务端内存中，相比浏览器端存储（如 `app.storage.browser`），支持存储非序列化对象（如数据库连接、流对象），且容量无浏览器限制；
- 轻量临时存储**：数据随标签页生命周期终结（关闭标签页）而自动清理，无需手动管理，避免内存泄漏。

2. 设计目标

- 解决“同一用户多标签页数据隔离”需求（如多标签页分别操作不同任务、独立登录状态）；
- 支持存储资源密集型或非序列化对象（如数据库连接、WebSocket 连接），避免浏览器存储的序列化限制；
- 平衡“数据保留”与“自动清理”：标签页活跃期间保留数据，关闭后自动释放，兼顾可用性与资源效率；
- 适配需要短期稳定存储但无需持久化到磁盘的场景（如临时会话数据、标签页级缓存）。

二、核心特性与关键规则

1. 存储基础信息

特性	详细说明
存储位置	服务器端内存（非磁盘文件），不写入本地 JSON 或浏览器 Cookie
数据支持	可存储任意 Python 对象（基础类型、自定义类实例、数据库连接、流对象等），无需序列化
生命周期	标签页会话周期：标签页打开时创建，关闭时销毁；服务器重启后，若标签页未关闭且重新连接，数据可保留
共享范围	仅当前标签页可见，不跨标签页、不跨客户端、不跨浏览器
依赖条件	需先建立客户端连接（通过 <code>await client.connected()</code> 确保连接就绪），否则无法操作

2. 关键使用限制

- **连接依赖**：必须在客户端与服务器建立连接后才能操作 `app.storage.tab`，否则会抛出错误。需通过 `await ui.context.client.connected()` 或 `await client.connected()` 等待连接就绪；
- **内存存储特性**：服务器重启时，若标签页未关闭，数据会暂时丢失，需等待 #2841 特性实现完全持久化；关闭标签页后，数据会被服务器自动清理；
- **无跨浏览器支持**：仅同一浏览器的同一标签页可用，切换浏览器、复制标签页（新标签页会创建新的 `tab` 存储）均无法共享；
- **最大保留时长**：默认标签页存储数据保留 30 天（从数据最后更新时间开始计算），可通过 `app.storage.max_tab_storage_age` 自定义时长。

三、核心配置与基础用法

1. 前置条件：确保客户端连接

操作 `app.storage.tab` 前，必须等待客户端与服务器建立连接，否则会因连接未就绪导致操作失败。常用两种方式实现：

- 方式 1：在页面函数中使用 `await ui.context.client.connected()`（推荐，无需传递 `client` 参数）；
- 方式 2：通过 `@app.on_connect` 装饰器，在客户端连接时操作（需接收 `client` 参数）。

2. 基础操作 API

`app.storage.tab` 的 API 完全模仿 Python 字典（dict），支持键值对的增、删、改、查，核心操作如下：

操作	语法	功能描述	示例
读取值	<code>app.storage.tab[key]</code> 或 <code>app.storage.tab.get(key, default)</code>	根据键读取值， <code>get</code> 方法支持默认值（键不存在时返回）	<code>count = app.storage.tab.get('count', 0)</code>
写入 / 更新值	<code>app.storage.tab[key] = value</code>	写入新键值对或更新已有键的值	<code>app.storage.tab['db_conn'] = create_db_connection()</code>
删除值	<code>del app.storage.tab[key]</code> 或 <code>app.storage.tab.pop(key, default)</code>	删除指定键， <code>pop</code> 方法支持默认值	<code>app.storage.tab.pop('temp_data', None)</code>

操作	语法	功能描述	示例
检查键存在	<code>key in app.storage.tab</code>	判断键是否存在	<code>if 'session_id' in app.storage.tab: ...</code>
清空存储	<code>app.storage.tab.clear()</code>	删除当前标签页的所有存储数据	<code>app.storage.tab.clear()</code>
获取长度	<code>len(app.storage.tab)</code>	返回当前标签页存储的键值对数量	<code>data_count = len(app.storage.tab)</code>

3. 自定义标签页存储保留时长

默认标签页存储数据保留 30 天（无操作时），可通过 `app.storage.max_tab_storage_age` 设置自定义时长（单位：秒），超过时长无操作的标签页数据会被自动清理：

```
from datetime import timedelta
from nicegui import app, ui

# 设置标签页存储保留 1 分钟（60 秒）
app.storage.max_tab_storage_age = timedelta(minutes=1).total_seconds()

@ui.page('/')
def index():
    ui.label(f'当前标签页存储保留时长: {app.storage.max_tab_storage_age} 秒')

ui.run()
```

四、典型应用场景与完整示例

场景 1：标签页级计数器（多标签页独立计数）

需求：同一用户打开多个标签页，每个标签页的计数器独立递增，页面重载后计数不丢失，关闭标签页后计数重置。

```
from nicegui import app, ui

@ui.page('/')
async def index():
    # 等待客户端连接就绪（必须步骤）
    await ui.context.client.connected()

    # 初始化计数器（键不存在时设为 0，存在时自增）
```

```

app.storage.tab['count'] = app.storage.tab.get('count', 0) + 1

# 展示当前标签页的计数
count_label = ui.label(f'当前标签页重载次数: {app.storage.tab["count"]}')

# 刷新按钮（重载页面，计数仍保留）
ui.button('重载页面', on_click=ui.navigate.reload)

# 重置按钮（清空当前标签页的计数器）
def reset_count():
    app.storage.tab['count'] = 0
    count_label.set_text(f'当前标签页重载次数: {app.storage.tab["count"]}')
    ui.button('重置计数', on_click=reset_count)

ui.run()

```

效果：打开多个标签页，每个标签页的计数独立递增；重载任意标签页，计数继续累加；关闭标签页后重新打开，计数从 1 开始。

场景 2：存储标签页专属资源（如数据库连接）

需求：每个标签页创建独立的数据库连接，用于动态数据更新，页面关闭后连接自动释放（无需手动关闭）。

```

from nicegui import app, ui
import sqlite3 # 示例：SQLite 数据库

@ui.page('/')
async def index():
    await ui.context.client.connected()

    # 若当前标签页未创建数据库连接，新建连接并存储
    if 'db_conn' not in app.storage.tab:
        # 创建数据库连接（标签页专属，其他标签页不可见）
        conn = sqlite3.connect('demo.db')
        app.storage.tab['db_conn'] = conn
        ui.notify('标签页专属数据库连接已创建')

    # 使用连接查询数据（示例：查询用户表）
    def query_data():
        conn = app.storage.tab['db_conn']
        cursor = conn.cursor()
        cursor.execute('SELECT name FROM sqlite_master WHERE type="table"')
        tables = cursor.fetchall()
        ui.notify(f'当前数据库表: {tables}')

    ui.button('查询数据库（标签页专属连接）', on_click=query_data)

    # 手动关闭连接（可选，关闭标签页后会自动释放）
    def close_conn():
        if 'db_conn' in app.storage.tab:
            app.storage.tab['db_conn'].close()
            del app.storage.tab['db_conn']

```

```
        ui.notify('标签页专属数据库连接已关闭')
        ui.button('关闭数据库连接', on_click=close_conn)

    ui.run()
```

优势：避免多标签页共享数据库连接导致的并发问题，标签页关闭后连接自动释放，节省服务器资源。

场景 3：标签页级临时会话数据（如独立搜索条件）

需求：每个标签页保存独立的搜索条件，切换标签页后搜索条件不干扰，页面重载后仍保留上次搜索条件。

```
from nicegui import app, ui

@ui.page('/')
async def index():
    await ui.context.client.connected()

    # 初始化搜索条件（从标签页存储中读取，无则设为空）
    default_query = app.storage.tab.get('search_query', '')

    # 搜索输入框（绑定标签页存储的搜索条件）
    search_input = ui.input('输入搜索条件', value=default_query)

    # 搜索按钮（保存搜索条件到标签页存储）
    def search():
        app.storage.tab['search_query'] = search_input.value
        ui.notify(f'标签页专属搜索条件已保存: {search_input.value}')

    ui.button('搜索', on_click=search)

    # 展示当前标签页的历史搜索条件
    if 'search_query' in app.storage.tab:
        ui.label(f'上次搜索条件: {app.storage.tab["search_query"]}')

    ui.run()
```

效果：每个标签页的搜索条件独立保存，切换标签页后搜索输入框显示各自的历史条件，重载页面后仍保留。

五、与其他存储类型的核心区别

为明确 `app.storage.tab` 的适用边界，以下对比 NiceGUI 其他四种存储类型的差异：

对比维度	.tab	.client	.user	.browser	.general
存储位置	服务器内存	服务器内存	服务器（文件 / Redis）	浏览器 Cookie	服务器（文件 / Redis）
跨标签页	否（独立）	否（独立）	是（共享）	是（共享）	是（共享）
跨浏览器	否	否	否	否	是

对比维度	.tab	.client	.user	.browser	.general
跨服务器重启	是（标签页未关）	否	是	否	是
跨页面重载	是	否	是	是	是
需客户端连接	是	否	否	否	否
数据类型支持	任意 Python 对象	任意 Python 对象	序列化类型	序列化类型	序列化类型
需 storage_secret	否	否	是	是	否

关键选型建议

- 需多标签页独立存储 → 选 `app.storage.tab`；
- 需页面访问期间临时存储（重载即丢） → 选 `app.storage.client`；
- 需用户级跨标签页共享（如用户偏好） → 选 `app.storage.user`；
- 需浏览器端存储（无服务器依赖） → 选 `app.storage.browser`（不推荐，优先 `user`）；
- 需全局所有用户共享（如应用配置） → 选 `app.storage.general`。

六、最佳实践与注意事项

1. 最佳实践

- **必等连接就绪**：所有操作 `app.storage.tab` 的代码，必须放在 `await client.connected()` 之后，否则会因连接未建立导致错误；
- **存储资源型对象**：优先用于存储数据库连接、流对象、WebSocket 连接等资源密集型或非序列化对象，发挥服务器端内存存储的优势；
- **避免长期持久化**：若需长期保存数据（如用户配置），应使用 `app.storage.user` 或 `general`，而非 `tab`（关闭标签页即丢失）；
- **合理设置保留时长**：根据业务需求调整 `max_tab_storage_age`，避免长期无操作的标签页数据占用服务器内存（如短期任务设为 5 分钟）。

2. 注意事项

- **服务器重启影响**：当前版本服务器重启后，标签页数据可能暂时丢失，需等待 `#2841` 特性完全实现后才能实现完整持久化；
- **内存占用风险**：若大量用户打开多个标签页且存储大型对象，可能导致服务器内存占用过高，需合理控制存储数据大小；
- **无自动序列化**：数据存储在服务端内存中，未持久化到磁盘，服务器崩溃时未关闭标签页的数据会丢失；
- **复制标签页行为**：复制标签页会创建新的 `tab` 存储实例，原标签页的数据不会复制到新标签页（完全独立）。

七、总结

`app.storage.tab` 是 NiceGUI 针对“标签页级独立存储”场景的专属解决方案，核心优势在于**标签页数据隔离、支持非序列化对象、服务器端存储、自动生命周期管理**。其适用场景集中在需要多标签页独立状态（如计数器、会话数据）、存储资源密集型对象（如数据库连接）的场景，且无需长期持久化到磁盘。

使用关键在于：确保客户端连接就绪后再操作、合理设置数据保留时长、避免存储需长期保留的数据。与其他存储类型相比，`app.storage.tab` 填补了“标签页级独立、服务器端临时存储”的空白，是多标签页应用中实现数据隔离的核心工具。